Name : Malaviya Parth Mahendrabhai

Roll no : CE027

ID : 24CEUOG067

Division : A2

Semester : V

Batch : 2024 - 28

Subject : SDP

Lab : 02

1. Dart Revision: Comments, Output, Statements and Expressions

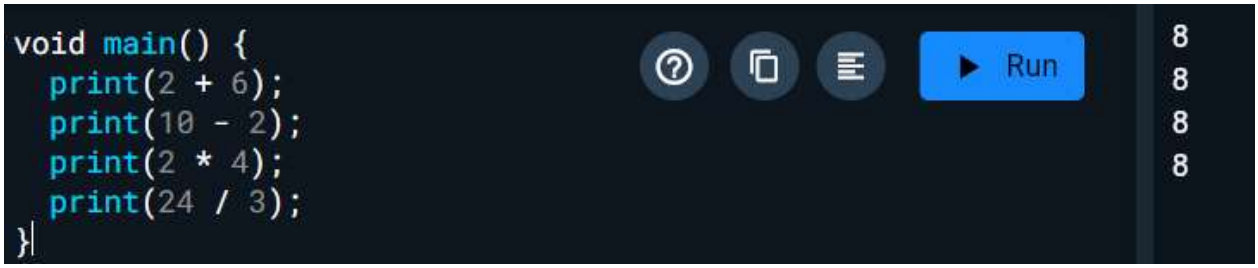
```dart
import 'dart:math';

/// Demonstrates comments, output, statements,
/// expressions and arithmetic operations in Dart.
void main() {

  // Single-line comment

  /*
    Multi-line comment
    This program covers almost every concept from Chapter 1.
  */

  // Variables (Statements)
  int x = 22, y = 7;

  // Expression
  int sum = x + y;

  // Printing strings
  print("Welcome to Dart!");

  // Printing variables
  print("x = $x");
  print("y = $y");

  // Printing expression directly
  print("x + y = ${x + y}");

  // Printing variable
  print("Sum = $sum");

  // Arithmetic Operators
  print("Addition       : ${x + y}");
  print("Subtraction    : ${x - y}");
  print("Multiplication : ${x * y}");
  print("Division       : ${x / y}");
  print("Integer Divide : ${x ~/ y}");
  print("Modulo         : ${x % y}");

  // Order of operations
  print("350 / 5 + 2 = ${350 / 5 + 2}");
  print("350 / (5 + 2) = ${350 / (5 + 2)}");

  // Math Library
  print("PI = $pi");
  print("sqrt(2) = ${sqrt(2)}");
  print("max(10,20) = ${max(10, 20)}");
  print("min(10,20) = ${min(10, 20)}");

  // Trigonometry (Angles in Radians)
  double angle = 45 * pi / 180;

  print("sin(45°) = ${sin(angle)}");
  print("cos(45°) = ${cos(angle)}");

  // Verification
  double value1 = 1 / sqrt(2);
  double value2 = sin(angle);

  print("1/sqrt(2) = $value1");
  print("Difference = ${value1 - value2}");
}
```

```
===== Dart Fundamentals =====
Welcome to Dart!
x = 22
y = 7
x + y = 29
Sum = 29
Addition       : 29
Subtraction    : 15
Multiplication : 154
Division       : 3.142857142857143
Integer Divide : 3
Modulo         : 1
350 / 5 + 2 = 72
350 / (5 + 2) = 50
PI = 3.141592653589793
sqrt(2) = 1.4142135623730951
max(10,20) = 20
min(10,20) = 10
sin(45°) = 0.7071067811865475
cos(45°) = 0.7071067811865476
1/sqrt(2) = 0.7071067811865475
Difference = 0
```

Mini Exercises

1. Print your name, department and semester using separate print() statements.

2. Print the values of 2 + 6, 10 - 2, 2 * 4 and 24 / 3.



3. Print the value of 22 / 7 and 22 ~/ 7. Explain the difference.

4. Print the value of 28 % 10. Explain why the answer is 8.

5. Print the value of 1 / sqrt(2) and confirm that it is approximately equal to sin(45 * pi / 180).

2. Variables, Identifiers, Constants and Scope

```dart
void main() {

  // Explicit Type
  int age = 20;
  double cgpa = 8.96;
  String name = "Parth";
  bool isStudent = true;

  // var (Type Inference)
  var city = "Junagadh";
  var marks = 567;
  var percentage = 94.5;

  // Mutable Variable
  age = 21;

  // final (Runtime Constant)
  final currentTime = DateTime.now();

  // const (Compile-Time Constant)
  const pi = 3.14159;
  const college = "DDU";

  // Printing Values
  print("Name         : $name");
  print("Age          : $age");
  print("CGPA         : $cgpa");
  print("Student      : $isStudent");
  print("\nCity         : $city");
  print("Marks        : $marks");
  print("Percentage   : $percentage");
  print("\nRuntime Types");
  print(city.runtimeType);
  print(marks.runtimeType);
  print(percentage.runtimeType);
  print("\nFinal: $currentTime");
  print("\nConst");
  print(pi);
  print(college);

  // Scope
  print("\nVariable Scope");

  String department = "Computer Engineering";

  if (true) {
    String semester = "Semester 5";

    print(department);
    print(semester);
  }

  print(department);

  // print(semester); // Error (Outside Scope)
}
```

```
Name         : Parth
Age          : 21
CGPA         : 8.96
Student      : true

City         : Junagadh
Marks        : 567
Percentage   : 94.5

Runtime Types
String
int
double

Final 2026-07-25 22:03:23.524

Const
3.14159
DDU

Variable Scope
Computer Engineering
Semester 5
Computer Engineering
```

Mini Exercises

1. Declare a constant myAge and set it equal to your age.

2. Declare a variable dogs and set it equal to the number of dogs you own. Increase it by one and print the new value.

3. Create two constants age1 = 42 and age2 = 21. Check that Dart infers both as int.

4. Create a constant averageAge as (age1 + age2) / 2. Check its type and explain why the result is double.

`/` always returns a double, even if both operands are `int`.

5. Declare three double constants rating1, rating2 and rating3. Calculate their average.

3. Data Types, Type Conversion and Operators

```dart
void main() {
  int age = 20;
  print(age);
  double cgpa = 8.96;
  print(cgpa);
  num value = 100;
  value = 99.99;
  print(value);

  String name = "Parth";
  print(name);
  bool isStudent = true;
  print(isStudent);

  dynamic data = 10;
  print(data);
  print(data.runtimeType);
  data = "Hello Dart";
  print(data);
  print(data.runtimeType);

  Object obj = 3.14;
  print(obj.runtimeType);

  num number = 12.5;
  print(number is double);
  print(number is int);
  print(number.runtimeType);

  double decimal = 12.5;
  int integer = decimal.toInt();
  print(integer);
```

```
20
8.96
99.99
Parth
true
10
int
Hello Dart
String
double
true
false
double
12
100
25.5
double
true
false
false
true
true
true
true
a is Greater
```

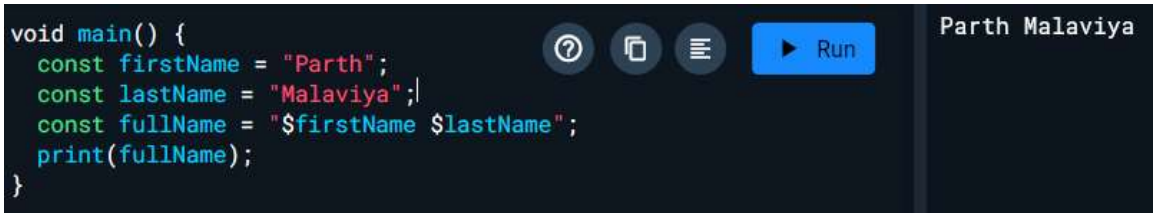
```dart
  String s1 = "100";
  int intValue = int.parse(s1);
  print(intValue);
  String s2 = "25.5";
  double doubleValue = double.parse(s2);
  print(doubleValue);

  int x = 10;
  double y = 5.5;
  print((x + y).runtimeType);

  int a = 17, b = 5;

  print(a > b);
  print(a < b);
  print(a == b);
  print(a != b);

  print(a > 10 && b < 10);
  print(a < 10 || b < 10);
  print(!(a == b));

  String result = (a > b) ? "a is Greater" : "b is Greater";
```

Mini Exercises

1. Create constants age1 = 42 and age2 = 21. Confirm the inferred type.

2. Create averageAge = (age1 + age2) / 2. Explain why the result is double.

3. Create constants firstName and lastName. Create fullName by joining them with a space.



4. Create a constant myDetails using string interpolation.

5. Convert a double into int using toInt() and observe loss of precision.

6. Use is and runtimeType to check the type of a value at runtime.

```dart
void main() {
  var value = 25.5;
  print(value is double);
  print(value is int);
  print(value.runtimeType);
}
```

```
true
false
double
```

4. Control Flow and Decision Making

```dart
enum AudioState { playing, paused, stopp }

void main() {

  String language = "Dart";
  print(language == "Dart");
  print(language == "Java");

  int a = 20, b = 10;
  if (a > b) print("a is greater");

  if (a % 2 == 0) print("Even");
  else print("Odd");

  int marks = 82;
  if (marks >= 90) {
    print("Grade A");
  } else if (marks >= 75) {
    print("Grade B");
  } else if (marks >= 60) {
    print("Grade C");
  } else {
    print("Fail");
  }

  String result = (marks >= 35) ? "Pass" : "Fail";
  print(result);
```

```
true
false
a is greater
Even
Grade B
Pass
Start of Week
Paused
```

```dart
  String day = "Monday";
  switch (day) {
    case "Monday":
      print("Start of Week");
      break;

    case "Friday":
      print("Weekend Coming");
      break;

    default:
      print("Normal Day");
  }
  AudioState state = AudioState.paused;
  switch (state) {
    case AudioState.playing:
      print("Playing");
      break;
```

Mini Exercises

1. Create a constant myAge. Create isTeenager using Boolean logic to check whether the age is between 13 and 19.

2. Create maryAge = 30. Create bothTeenagers to check whether both you and Mary are teenagers.

3. Create string constants reader and ray. Create a Boolean rayIsReader using string equality.

4. Write an if statement to print Teenager or Not a teenager.

5. Replace the previous if-else with a ternary conditional operator and store the result in answer.

```dart
void main() {
  const myAge = 20;
  const answer =
      (myAge >= 13 && myAge <= 19) ? "Teenager" : "Not a teenager";
  print(answer);
}
```

```
Not a teenager
```

6. Make an enum AudioState with values playing, paused and stopped.

```dart
enum AudioState { playing, paused, stopped }

void main() {
  AudioState state = AudioState.playing;
  print(state);
}
```

```
AudioState.playing
```

5. Loops and Repetition

```dart
import 'dart:math';

void main() {

  int i = 1;
  while (i <= 3) {
    print("while: $i");
    i++;
  }

  int j = 1;
  do {
    print("do-while: $j");
    j++;
  } while (j <= 3);

  for (int k = 1; k <= 3; k++) {
    print("for: $k");
  }

  for (int n = 1; n <= 5; n++) {
    if (n == 4) break;
    print("break: $n");
  }

  for (int n = 1; n <= 5; n++) {
    if (n == 3) continue;
    print("continue: $n");
  }

  Random random = Random();
  print("Random: ${random.nextInt(10)}");
```

```
while: 1
while: 2
while: 3
do-while: 1
do-while: 2
do-while: 3
for: 1
for: 2
for: 3
break: 1
break: 2
break: 3
continue: 1
continue: 2
continue: 4
continue: 5
Random: 4
Apple
Banana
Mango
10
20
30
```

```dart
List<String> fruits = ["Apple", "Banana", "Mango"];
for (String fruit in fruits) {
  print(fruit);
}

List<int> numbers = [10, 20, 30];
numbers.forEach((number) {
  print(number);
});
```

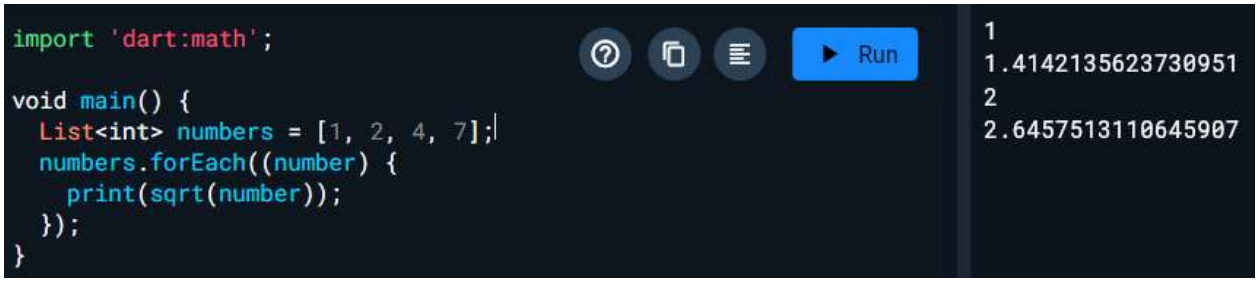
Mini Exercises

1. Create a variable counter = 0. Use a while loop with condition counter < 10. Print counter is X and increment counter.

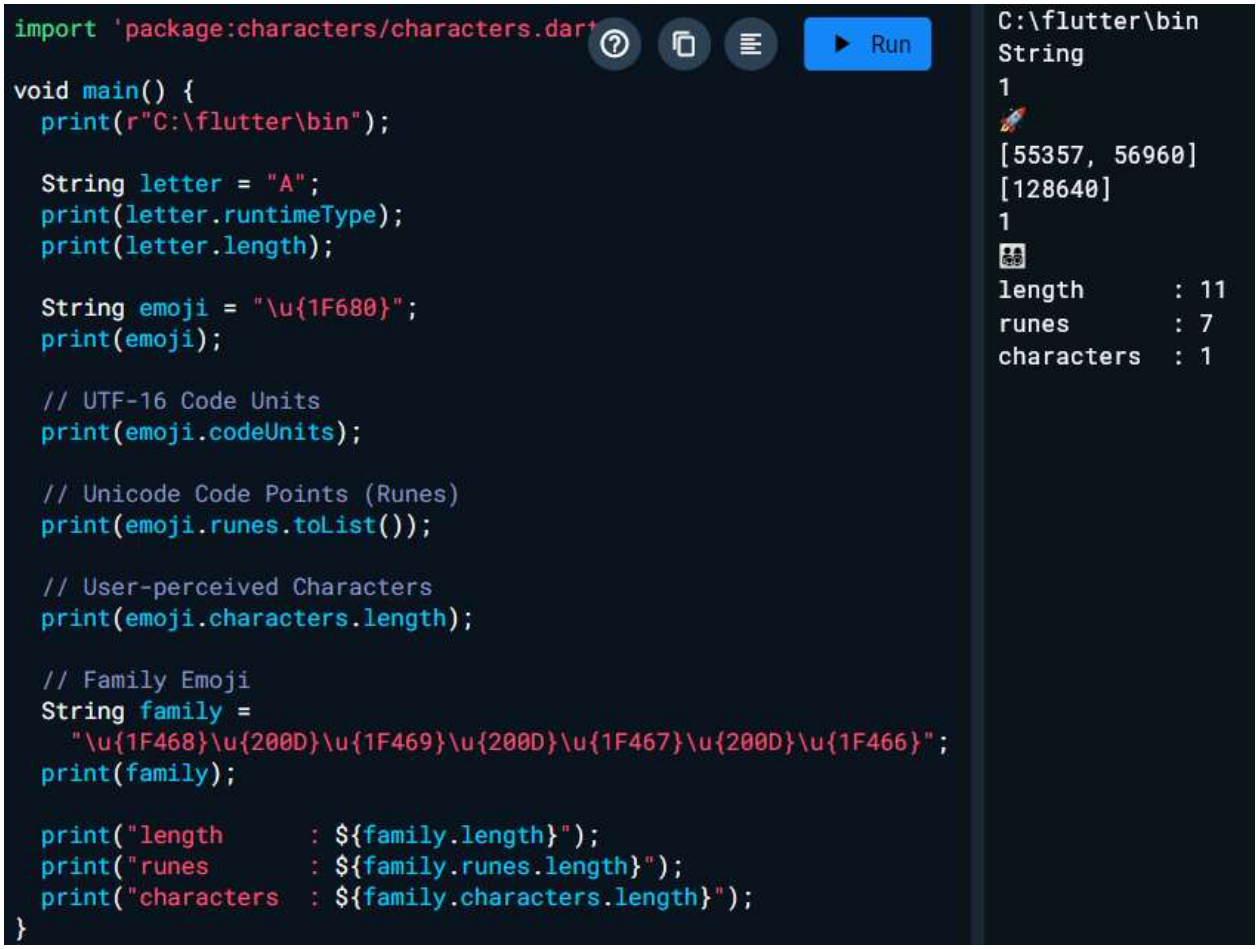
2. Write a for loop from 1 to 10 inclusive and print the square of each number.

3. Write a for-in loop over [1, 2, 4, 7] and print the square root of each number.

4. Repeat the previous exercise using forEach.



6. Flutter Strings, Unicode and Character Encoding



```dart
import 'package:characters/characters.dart';

void main() {
  print(r"C:\flutter\bin");

  String letter = "A";
  print(letter.runtimeType);
  print(letter.length);

  String emoji = "\u{1F680}";
  print(emoji);

  // UTF-16 Code Units
  print(emoji.codeUnits);

  // Unicode Code Points (Runes)
  print(emoji.runes.toList());

  // User-perceived Characters
  print(emoji.characters.length);

  // Family Emoji
  String family =
    "\u{1F468}\u{200D}\u{1F469}\u{200D}\u{1F467}\u{200D}\u{1F466}";
  print(family);

  print("length      : ${family.length}");
  print("runes       : ${family.runes.length}");
  print("characters  : ${family.characters.length}");
}
```

```
C:\flutter\bin
String
1
🚀
[55357, 56960]
[128640]
1
👨‍👩‍👧‍👦
length      : 11
runes       : 7
characters  : 1
```

String, Unicode and Encoding Exercises

1. Write a Flutter-style Text snippet that displays English, Gujarati, Hindi and one emoji.

```
Column(
  children: const [
    Text("Hello"),
    Text("નમસ્તે"),
    Text("हिन्दी"),
    Text("😀"),
  ],
);
```

2. Write a Dart program to demonstrate string interpolation.

```dart
void main() {
  String name = "Parth";
  print("My name is $name");
}
```

```
My name is Parth
```

3. Print the UTF-16 code units of Hello!.

```
void main() {
  print("Hello!".codeUnits);
}
```

Run

```
[72, 101, 108, 108, 111, 33]
```

4. Print the runes of the dartboard emoji.

```
void main() {
  String dartboard = "\u{1F3AF}";
  print(dartboard.runes.toList());
}
```

Run

```
[127919]
```

5. Compare length, runes.length and characters.length for a complex emoji.

```
import 'package:characters/characters.dart';
void main() {
  String family =
    "\u{1F468}\u{200D}\u{1F469}\u{200D}\u{1F467}\u{200D}\u{1F466}";
  print(family.length);
  print(family.runes.length);
  print(family.characters.length);
}
```

Run

```
11
7
1
```

6. Encode a multilingual string using UTF-8 and decode it again.

```
import 'dart:convert';
void main() {
  String text = "Hello નમસ્તે 😀";
  List<int> bytes = utf8.encode(text);
  print(bytes);
  String decoded = utf8.decode(bytes);
  print(decoded);
}
```

Run

```
[72, 101, 108, 108, 111, 32, 224, 170, 168, 224, 170, 174, 224,
170, 184, 224, 171, 141, 224, 170, 164, 224, 171, 135, 32, 240,
159, 152, 128]
Hello નમસ્તે 😀
```

7. Explain why String.length may be 2 for one emoji.

Dart stores strings as UTF-16 code units. Many emojis require 2 UTF-16 code units (surrogate pair). Therefore: "😀".length returns 2 even though only one emoji is visible.

8. Explain the difference among Unicode, code point, code unit and grapheme cluster.

Term	Meaning
Unicode	Standard that assigns a unique number to every character.
Code Point	The Unicode number (e.g., 😀 = U+1F600).
Code Unit	Storage unit used by an encoding (UTF-16 uses 16-bit units).
Grapheme Cluster	One visible character seen by the user (may contain multiple code points).

9. Compare UTF-8, UTF-16 and UTF-32 in terms of unit size, variable length and common use.

UTF Comparison

Encoding	Unit Size	Variable Length	Common Use
UTF-8	8 bits	Yes (1–4 bytes)	Web, APIs, Files
UTF-16	16 bits	Yes (1 or 2 units)	Dart Strings, Windows
UTF-32	32 bits	No (always 1 unit/code point)	Internal processing

10. Write a Flutter form validator that counts visible characters using the characters package.

```
import 'package:characters/characters.dart';

String? validateName(String? value) {
  if (value == null || value.characters.length < 3) {
    return "Enter at least 3 characters";
  }

  return null;
}
```

11. Write notes on how Dart works with characters.

Dart stores text using UTF-16.

There is no `char` type; every character is a `String`.

`length` counts UTF-16 code units.

`runes` returns Unicode code points.

`codeUnits` returns UTF-16 code units.

`characters` counts visible characters (grapheme clusters).

12. Explain surrogate pairs with a suitable example.

Concept Words to Explain

1. Comment

Definition: Used to explain code. Ignored by the compiler.

// This is a comment

2. Documentation Comment

Definition: Used to document classes, methods, and libraries.

/// Prints a welcome message.

3. `print()`

Definition: Displays output on the console.

print("Hello");

4. Statement

Definition: A complete instruction executed by the program.

int age = 20;

5. Expression

Definition: Produces a value.

10 + 20

6. Operator

Definition: Performs an operation on operands.

+

7. Operand

Definition: Values on which an operator works.

10 + 20

Operands: `10`, `20`

8. Identifier

Definition: Name of a variable, function, class, etc.

int marks = 90;

Identifier: `marks`

9. Keyword

Definition: Reserved word with special meaning.

int
if
class
return

10. Variable

Definition: Stores data that can change.

int age = 20;

11. Constant

Definition: Value that cannot be changed.

const pi = 3.14;

12. `final`

Definition: Assigned once at runtime.

final time = DateTime.now();

13. `const`

Definition: Compile-time constant.

const pi = 3.14;

14. Type Inference

Definition: Dart automatically determines the variable type.

var city = "Junagadh";

15. Runtime Type

Definition: Returns the actual type of an object.

print(10.runtimeType);

Output:

int

16. `int`

Definition: Stores whole numbers.

int age = 20;

17. `double`

Definition: Stores decimal numbers.

double cgpa = 8.96;

18. `num`

Definition: Can store both `int` and `double`.

num value = 10.5;

19. `String`

Definition: Stores text.

String name = "Parth";

20. `bool`

Definition: Stores `true` or `false`.

bool passed = true;

21. `dynamic`

Definition: Variable whose type can change.

dynamic value = 10;
value = "Hello";

22. `Object`

Definition: Parent type of most Dart objects.

Object obj = 3.14;

23. Type Conversion

Definition: Converts one type to another.

double d = 12.5;
int i = d.toInt();

24. Type Casting

Definition: Treats an object as another compatible type.

Object obj = "Hello";
String text = obj as String;

25. Boolean Logic

Definition: Uses logical operators (&&, ||, !).

```
print(true && false);
```

26. Scope

Definition: Region where a variable is accessible.

```
if (true) {
  int x = 10;
}
```

27. `if` Statement

Definition: Executes code when a condition is true.

```
if (age >= 18) {
  print("Adult");
}
```

28. `switch` Statement

Definition: Selects one block from multiple choices.

```
switch(day){
  case "Mon":
    print("Monday");
}
```

29. `enum`

Definition: Defines a fixed set of named values.

```
enum Color { red, green, blue }
```

30. `while` Loop

Definition: Repeats while the condition is true.

```
while(i < 5){
  i++;
}
```

31. `do-while` Loop

Definition: Executes at least once.

```
do{
  i++;
}while(i < 5);
```

32. `for` Loop

Definition: Repeats a fixed number of times.

```
for(int i=1;i<=5;i++){
  print(i);
}
```

33. `break`

Definition: Exits the loop immediately.

```
if(i==5) break;
```

34. `continue`

Definition: Skips the current iteration.

```
if(i==5) continue;
```

35. `for-in` Loop

Definition: Iterates through a collection.

```
for(var item in list){
  print(item);
}
```

36. `forEach`

Definition: Executes a function for every element in a collection.

```
list.forEach((item){
  print(item);
});
```

37. Unicode

Definition: Universal standard that assigns a unique number to every character.

Example:

A → U+0041
😀 → U+1F600

38. UTF-8

Definition: Unicode encoding using 1–4 bytes. Common for web and files.

39. UTF-16

Definition: Unicode encoding using 1 or 2 sixteen-bit code units. Dart strings use UTF-16.

40. Code Unit

Definition: Smallest storage unit of an encoding.

```
print("A".codeUnits);
```

Output:

```
[65]
```

41. Rune

Definition: Unicode code point in Dart.

print("😀".runes.toList());

Output:

[128512]

42. Surrogate Pair

Definition: Two UTF-16 code units representing one Unicode code point above U+FFFF.

Example:

print("🚀".length);

Output:

2

43. Grapheme Cluster

Definition: A user-perceived character. It may consist of one or more Unicode code points.

Example:

import 'package:characters/characters.dart';

print("👨‍👩‍👧".characters.length);

Output:

1

Although it is made of multiple code points, it appears as one visible character.